

Healthy Oncall Practices

Oncall benefits and downsides, healthy approaches from well-known companies, and oncall regulations across countries.



GERGELY OROSZ

JUL 26, 2022 • PAID



84



9



Share



Q: “I want to improve our oncall process and decide if we should pay engineers for being oncall. How do companies approach this across the industry; from startups through Big Tech, to more ‘traditional’ companies?”

This is an excellent question and one that prompted me to go deep and broad on this topic, including talking with software engineers and engineering managers at almost 100 companies for details of how they do oncall.

This issue covers:

1. **What is oncall?** A short overview of how engineers carrying around pagers – by which I mean pager apps on smartphones – came about.
2. **The benefits of engineers being oncall.** It should be little surprise that there are a few.
3. **Oncall downsides.** There are plenty of these.
4. **Monoliths, microservices, tiers and oncall.** How are these connected?
5. **Oncall approaches.** Google, Intercom, LaunchDarkly, TravelPerk and Monzo: Five companies with decent oncall approaches. How do they organize it?
6. **Healthy oncall practices.** Examples of companies with decent oncall approaches, and ideas worth considering.

7. **Regulations across countries.** Which countries have oncall regulations worth knowing about?

This is part one of a two-part series about the huge topic of oncall. Next week, we will cover oncall philosophies across the industry and oncall compensation approaches – and numbers – from across more than 100 companies.

1. What is oncall?

Believe it or not, oncall has not always been as ubiquitous in the industry as it is today. Back in the 1990s and even the 2000s, many companies operated with the Ops model – Ops being short for “operations:”

1. Developers write the code, test it in their local environment, and commit it to a “next release” branch in the codebase.
2. Over some time – weeks or even months – a Release Candidate is finalized in this branch.
3. The Release Candidate is cut, and more tests might be run on it.
4. Operations takes over and ships the Release Candidate. For web apps, this means deploying the new code across all servers, and applying database schema changes. For downloadable applications, it means updating the binary, and ensuring the update scripts point to this new version.
5. Going forward, all alerts go to Operations, where the Operations team monitors the health of the application. If they see issues, they try to resolve them. If needed, they reach out to developers for help.

As continuous integration and deployment practices have spread, and as shorter iteration loops become more common, it's now rare for most startups, scaleups or Big Tech to wait weeks for a new release. Instead, some companies ship [directly into production](#), while some others perform extra validation steps. I cover [ways companies ship into production in-depth, in this article](#).

In most cases these days, the team monitoring the code is not Operations. It's the engineering team that makes the changes. Specifically, it's the oncall engineer who

gets alerts if anything goes wrong.

Being oncall means being on standby for alerts on systems you're responsible for. Let's start by describing how a common oncall setup works at a Big Tech company like Amazon or Uber.

There is a **primary oncall** defined for every system relevant for production. When an **alert** comes in through a **pager** – usually a pager app running on phones – the primary oncall **acknowledges** this alert, and then proceeds to verify if it points to a problem, or an **outage** as we'll call it in this article.

If the primary oncall doesn't acknowledge this alert; for example, they miss the notification for whatever reason, then the alert will most likely be **escalated** through the escalation chain to the **secondary oncall** within a few minutes. Should the secondary oncall also not acknowledge the alert, the third line of defense – called the **tertiary oncall** – is paged. This is either the engineering manager for the team, or an SRE (Site Reliability Engineer.) Companies like Amazon have further escalation paths which go up through the management chain, all the way to the CEO.

2. The benefits of engineers being oncall

All of this has potential to cause a lot of disruption, especially outside of working hours, and doubly so if it's the dead of night. So what is the benefit of software engineers being oncall, versus having a dedicated team handling all alerts across the company? There are several advantages to engineers being oncall for the systems they own:

- **Outages are resolved faster.** If something goes wrong with a system, how do you fix the issue quicker? Option one: notify someone with no context about the system. Option two: notify the engineer who built much of the system, who can likely debug and fix the issue quickly? By teams owning their oncall, the pager goes to someone with plenty of context, who can resolve outages quicker, without going through an escalation chain.
- **Better customer experience.** Companies where outages get resolved quickly – we're talking in minutes or hours, versus days – provide a better customer

experience. A superior customer experience makes it easier to retain customers. Frequent or long outages, on the other hand, lead to customer churn.

- **"You build it, you own it."** Oncall promotes strong ownership, with which teams can work with more autonomy than not. If a team owns their oncall, they have the freedom to choose the technology they build their service with, and make changes which a central SRE team – who would be oncall for their service – would reject. They can do whatever they want, *as long as the service runs reliably*. They can make decisions which increase the number of alerts and even outages: they have to live with the consequences of their decisions, after all!
- **Build faster, move faster.** Teams that own their oncall can deploy when they want – and they usually deploy immediately on code changes – and can make technological choices without seeking input from other teams. If another team owned oncall responsibilities, that team would be more cautious about making a move, thereby slowing down the shipping speed.
- **A faster feedback loop by "feeling the pain" of their own decisions.** I've talked with engineering teams where developers don't deploy their own code; Ops does it and Ops is then oncall for them. In these setups, Ops frequently complains developers don't prioritize resolving things that result in alerts or even outages, as they don't "feel the pain," Ops people do. By owning oncall, engineers feel the pain of their own decisions. For example, making a risky change without testing it properly which then causes an outage? The engineer making this change might be the one who wakes up to fix it; therefore the incentive exists to learn from the event and avoid it will be much stronger, and they fix the issue quicker.
- **More efficient knowledge-sharing and debugging.** Shared understanding of how systems work is always a challenge at companies, and this is especially true when onboarding new joiners. Oncall for all team members can help speed up this knowledge-sharing, as those going oncall need to have a good understanding of the system and the tools to debug and resolve outages.
- **In theory, prioritize a good oncall experience.** In theory, a team owning their own oncall would optimize the oncall experience as a healthy one. I say "in theory" as this should be the case, but both my own experience and discussions with

engineers across the industry show this does not happen without an explicit focus on it.

3. Oncall downsides

If oncall is all about benefits, we'd have no hesitation to make it part of every engineer's job. However, oncall comes with a long list of downsides:

Distraction and stress 24/7. Oncall typically means being ready to act on an alert for a whole week, around the clock, 24/7. This means carrying your laptop around outside of work, not going to places without mobile reception or data coverage, and additional stress if you're out of the office or not at home when you get an alert.

When I worked at Uber, I remember having drinks with the team in the evening when we got an alert, and the two oncall engineers started working on assessing the alert. As it was an outage, this turned into a mitigation exercise.

No option to opt out. At places where oncall is not voluntary, there's often no option of passing up oncall. The only option is to trade these oncalls.

If the oncall expectation was not communicated during the hiring process, this can cause misaligned expectations. There are also personal situations where some people would prefer to do no oncall for an extended period of time.

Lower productivity and burnout. I remember when one of my colleagues at Uber came to work after a night of repeated alerts. He was like a zombie, got no real work done, except for pushing code that later caused an outage.

This person was exhausted, and in hindsight was close to burning out due to little sleep and lots of stress, all from being oncall.

Pressure to do oncall as well as day-to-day work. Several engineers shared how they feel pressure to deliver on "regular" work while they are oncall, even if they get a swarm of alerts or have outages to deal with.

Such an expectation can lead to burnout. It also pushes people to under-prioritize oncall and systems reliability and to prioritize “regular” work more. In such situations, oncall alerts are more likely to be ignored, and people deprioritize work to make the oncall systems better.

Oncall: avoid copying what most of Big Tech does. Many startups look to the likes of Meta, Amazon, Microsoft and similarly large companies for guidance on which engineering practices to put in place. This is even more true if the founders previously worked at companies like these. And many of these practices are sensible; that’s a reason I do deep-dives on some of these companies, such as [on Amazon](#) and [on Facebook](#).

However, healthy oncall practices are rarely to be found at Big Tech, save perhaps for Google – on which we’ll go into more detail, below. Big Tech tends to push almost all engineers to be oncall, resulting in more oncall burnout than at companies which optimize for less oncall toil. Also, Big Tech typically does not compensate for oncall, neither in the form of cash payments nor vacation time.

Similarly to when [hiring software engineers](#) or [hiring and retaining diverse teams](#), startups and scaleups are better served to learn about oncall practices which other startups, scaleups – and even traditional companies! – employ. These are often healthier than what we observe at Big Tech.

Burnout is a downside I cannot overstate. I’ve talked with several engineers who shared how much they got burnt out when working at companies with oncalls entailing hundreds of alerts per week.

One of the companies about which I hear the most oncall related burnout stories is Amazon. While the company pays pretty well, it places a large focus on operational excellence – which I cover in-depth in [Inside Amazon’s Engineering Culture](#). It does this at the expense of engineer wellbeing. I talked with several former Amazon engineers who described how intense oncalls were in organizations like Redshift or Alexa, and the burnout they suffered. It’s not just Amazon, of course. However, Amazon is the company within the Big Tech group most likely to consistently choose reliability over engineer wellbeing, from all I’ve gathered.

4. Monoliths, microservices, tiers and oncall

Building microservices is an increasingly popular approach for startups, scaleups and some of Big Tech. The idea behind microservices is that any team can spin up their own services, and then own the operating of this service. Naturally, that team also own oncall.

Contrast this with the monolith approach, where the whole codebase is deployed in one service. All teams make changes to this large codebase, and the code gets deployed on a regular basis to production. Oncall for this monolith can then be shared across all the teams contributing to it, or it can be attended to by a dedicated team which does the deployments, like an SRE team.

Working with microservices almost always comes with larger overall oncall load and noise, versus that of a monolith. There are several advantages to working with microservices, the biggest being the autonomy of teams which are able to move fast and to build and deploy as they want, when they want. However, any organization building on top of microservices needs to expect the very teams building the microservices will cover oncall for those same services.

A problem with microservices is that spikes of traffic burst can trigger alerts on smaller services. And because many services are small, this makes for frequent alerts. This was a problem that customer data startup Segment faced in 2018, when they wrote about why they started to consolidate their microservices setup. [As they wrote](#):

"The first item on the list was to consolidate the now over 140 services into a single service. The overhead from managing all of these services was a huge tax on our team. We were literally losing sleep over it, since it was common for the oncall engineer to get paged to deal with load spikes."

Other problems include how it's hard to execute integration tests across microservices. For example, say that Service A – owned by Team A – makes an innocent change. This change then causes an outage in one of its consuming services – Service B, owned by Team B. In a microservice-heavy environment, where integration testing of a series of

services is challenging, such outages are common. This is a fancy way of saying that cascading failures are more common with microservices.

Tiering of microservices is an initiative companies naturally come to, after their number of services explodes. After all, there is a difference between what happens when a core service responsible for operating the main product is down, versus when a new, experimental service which has very little production traffic goes down.

At Uber, we did a tiering exercise around 2018, assigning tiers to each service. The tier determined the SLAs and SLOs the service needed to adhere to, which type of oncall the service needed to operate, and if this oncall was paid. The tiers were as follows:

- Tier 0: critical service powering most key services. Oncall needs to acknowledge in less than 5 minutes and includes dedicated oncall with short rotation, all paid.
- Tier 1: key service powering a core flow directly or indirectly. Oncall needs to acknowledge in 10 minutes and is paid.
- Tier 2: a service powering a user-facing experience. Oncall needs to acknowledge in 30 minutes and is not paid.
- Tier 3: a service powering a non-user facing experience. Oncall is "best effort" and not paid.

Oncall load can be a reason to consolidate a microservice architecture into "macro-services" or monoliths. With a microservices setup, the ability of teams to execute independently is traded off for the same teams and engineers to deal with high levels of operational load and the stress of oncall. It's usually hard to share oncall rotations between teams, as those teams don't share ownership across the same codebase, and don't have the expertise to revert changes in another codebase in order to mitigate outages.

When combining smaller services into one, larger service to which multiple teams contribute, the testability of this larger service improves and oncall can be shared across the contributing teams. With better testability, the number of outages is also likely to fall. The downside of this approach? It's harder for any individual team to move fast and ignore the rest of the group.

It should be little surprise that companies favoring microservice architecture face much higher oncall loads. This is true even if those companies don't acknowledge that microservices and autonomous "two-pizza" teams contribute to their own operational load. Companies like Amazon and Uber are perfect examples: both organizations are known for heavy oncall loads on teams which either own lots of small services, or are frequently hit by load spikes or cascading failures originating within other services.

Compensating for oncall would be a heavy expense for these companies, as they'd have to pay for practically every software engineer, as all teams operate several microservices, which they own oncall duties for.

Shopify, on the other hand, still operates its code in a single Ruby on Rails monolith, albeit [a modular one](#). This allowed the company to create a global resilience team with a follow-the-sun rotation that monitors their application's health, mitigates issues, and pulls in teams as needed – as Ryan McIlmoyl, formerly the developer manager on the Resiliency team has [explained](#).

5. Oncall approaches: Google, Intercom, LaunchDarkly, TravelPerk, Monzo

Google has invested plenty into reliability, but also into ensuring that oncall does not burn software engineers out. For reliability, Google arguably introduced the Site Reliability Engineer (SRE) role to the wider tech industry, and hastened the adoption of this role by publishing several books on site reliability engineering practices, which are [all available to read for free](#).

As an important step, Google staffs their SRE organization very well. Google employs more than 3,000 SREs for its circa 30,000 software engineers. SREs are grouped into product areas, of between 50 and 300 SREs each, [according to](#) Dr. Christof Leng, Google's SRE Engagements engineering lead. SREs build tooling which helps with everything reliability-related: from production deployments tooling, through monitoring and alerting tools.

Toil SLOs (Service Level Objectives) have been a key initiative that's helped Google's oncall experience become a healthy one. The number of pages a team receives is recorded as an SLO. If this SLO is breached, the team needs to stop production work and fix the systemic issue that caused the breach. For example, it's typical that if a team gets more than 10 pages in a week, the SLO is considered to be broken, and work needs to be done to resolve this.

Because these SLOs are enforced, non-noisy oncall is taken into account from when services are architected. Google's approach is very different from how Amazon approaches the same problem. Amazon focuses on operational excellence, but not on toiling the engineering lead.

Oncall tiers, response times and compensation being linked is another approach Google put in place. Each service needs to define the tier of oncall it needs. Tier 1 is reserved for key services, Tier 2 is for less critical ones, and Tier 3 is for everything else. Here is how response times and compensation is coupled:

Tier	Time to acknowledge pages	Compensation (if choosing cash)	Compensation (if choosing vacation)
Tier 1	5 minutes	66% of salary	40 minutes for every hour
Tier 2	30 minutes	33% of salary	20 minutes for every hour
Tier 3	Best effort	None	None

pragmaticengineer.com

Oncall tiers, expectations and compensation at Google.

Limiting time spent oncall is one more thing Google does, which few other places do. Engineers have a limit of two weeks of compensated oncall time per quarter, meaning you are not allowed to be oncall for time in excess of this payment limit of cash or vacation.

If this limit is reached, managers need to grant an exception for the engineer to do extra oncall. Granting this exception is not encouraged, to protect engineers from burning

out.

Combine the oncall limit with the Toil SLOs – which ensures oncalls are relatively quiet – and, as a Google engineer puts it:

“This means that you’re getting an additional 6-7 weeks’ of vacation or pay in exchange for sitting on the couch for 2-3 weekends a quarter watching TV, which is a fair deal.”

Minimum viable oncall sizes are also promoted across Google. This is around 14 engineers. Oncalls below this size are not recommended.

Where does the 14-person oncall size target come from? A team needs to cover for 128 hours per week to cover oncall outside business hours, with two rotations, primary and secondary. That’s 5.25 days per week, or 71.5 days per quarter, per rotation. No engineer is allowed to spend more than two weeks oncall in a quarter, and two weeks spent oncall mean 10.5 days oncall.

$71.5/10.5 = 7$. So 7 engineers are needed per rotation, and 14 for the whole rotation.

Note that if the team is distributed across locations, the oncall situation improves drastically. Take a team split between New York in the US, and Zurich in Switzerland. Taking into account the time zones of the US and Europe, it is only 6 hours per weekday (from 11pm-5am EST time) to cover, plus 48 hours per weekend. That’s 2.5 days per week spent oncall per rotation, versus the 5.25 per week in a single timezone. This means oncall is more than 50% “lighter” in a distributed setup, thanks to business hours overlapping between the teams.

Intercom is a customer communication scaleup with more than 25,000 organizations using it. The company has over 800 employees and was valued at \$1.3B during its last funding round in 2018. The company [shared](#) how it revamped its oncall setup in 2018, after noticing they were putting engineers at risk of burnout.

Intercom’s approach to how they did oncall before this change is typical to how many startups, scaleups – and most of Big Tech – work today. As Intercom wrote:

"New teams started building a lot of new features and services, and they assumed full oncall responsibilities. This felt natural at the time, as it was a lightweight way to scale our oncall team and was consistent with our values that emphasized the importance of ownership.

Without deliberately planning it, we had ended up with four or five teams that were regularly being paged out of hours. The remaining product teams had a handful of alarms that rarely, if ever, resulted in an out of hours page."

Intercom realized there were a few problems with this setup:

- **Too many people on call.** An engineer from every team was always oncall. This felt wasteful, when ensuring the company's services were up could have been done with fewer people.
- **Inconsistent quality.** Most teams used ad-hoc processes. Runbooks were mostly non-existent.
- **Inconsistent expectations.** One oncall was paid, the others not. Expectations of what being oncall meant varied for everyone.
- **Not for everyone.** Some people don't have the capacity to take on oncall outside of working hours. Yet, in the old setup there was no opt-out.

Creating a consolidated, virtual, volunteer-based oncall team is how Intercom changed its process in 2018, when they had about 150 engineers. This team then improved the oncall setup like this:

"The team agreed upon and defined what acceptable alarms and runbooks look like, and described an acceptance process for moving alarms over to the new oncall team. They defined all our alarms in code using a Terraform module, and started using peer review for every change.

We put in place a level of compensation that we were happy with for taking a week's worth of on call shifts. We also created a "Level 2" escalation team made up of engineering managers to be a single point of escalation for the oncall engineer."

A reason why most companies are reluctant to "centralize" oncall the way Intercom did, is the fear that ownership will drop and the number of alerts rise. However, this was the opposite of what Intercom experienced.

Any action that caused an unwarranted page resulted in engineering teams taking swift action, and the company saw the number of out-of-hours pages drop. I'd speculate this was due to the "central" team taking a closer look at oncall, and then likely improving the quality of alerts and reducing noise.

Reviewing every page the next day is something Intercom started to do recently, and it works well for them. As [Ryan Sherlock](#), engineering manager at Intercom shares:

"Every time a STRIKE engineer is paged, we create a P1 for it and as part of the process, the team that owns the page will perform a review the following day even if everything was resolved/fixed.

Where it gets more tricky is if there is a novel product defect - when there are no runbooks - that needs to be resolved. Then it's an escalation to a commander and a best effort in getting the right folks pulled in.

Such novel defects are really rare. For a typical engineer, they would only occur every few years. We'd also make sure that any engineers paged out of hours to help on something "feel whole" afterwards - usually via additional time off etc."

Formalizing engineering managers to whom issues can be escalated is another thing Intercom did. Also from Ryan:

"Years ago, we had issues with our Incident Command process. Until then, there was a group of senior leaders that would all get paged and whoever would respond would be the incident commander. This was too vague and we switched to something like STRIKE with an explicit on-call commander at all times.

We usually have six volunteers who are typically experienced engineering managers-or-above. If an incident is longer running, people management can come into play and you need to be experienced/comfortable with directing at times.

Incident Commanders get paid similar to STRIKE but get paged a lot less and these pages are almost always during office hours: unless a vendor has an outage outside these hours.

Then it's an escalation to a Commander and a best effort in getting the right folks pulled in. We spread context pretty well around engineers on teams so it's pretty easy to get the right context online.

As an Incident Commander, I've had many weeks on command without a page. To become a commander, we'd like folks to have been around a while so they know the lay of the land, and then we put them through training and buddying sessions. It's worked well so far, and we are continually tweaking our process."

LaunchDarkly ended up in a similar place to Intercom. [The company](#) pays \$1,250 for engineers to go on a voluntary 7-day shift, plus an additional \$500 when going oncall during US holidays. LaunchDarkly has more than 500 employees, about 100 of whom are software engineers. The company was valued at \$3B and has more than 3,000 customers relying on its feature flagging product.

Software engineer [Anna Baker](#) works at LaunchDarkly and shared more details on how oncall works there.

"As LaunchDarkly has engineers in both the US and the UK, it does follow-the-sun oncall during weekdays, and the oncall engineers are only oncall during evening hours in the US for 7 hours, before the UK wakes up. During weekends, this is a 24 hour shift.

The company originally started off with one oncall rotation, and has now split this into two rotations, where each rotation owns a broad range of services and can mitigate issues which arise. The company also set up escalations from one rotation to go to the other. So far, this setup works well.

In any given quarter, there are 6-7 engineers on a rotation, which translates to an average of 2 weeks oncall. Every oncall shift has two engineers, plus one incident manager. So about 14 engineers of the ~100 are oncall over a quarter."

The possibility of a volunteer-only oncall was a risk. Anna shared they were worried about an uneven split when they relied only on volunteers, but that's not how it played out thanks to some incentives put in place:

"One of the risks we saw going into this model was that the volunteer pool would be unevenly spread, resulting in an unfair distribution of load. We've mitigated this by explicitly offering the opportunity to step down from oncall, as well as to on-board, every couple quarters. We've consistently had more volunteers than we've had the capacity to onboard, which has defied even our most optimistic predictions."

The Co-pilot program is unique to LaunchDarkly. When a new engineer onboards to the rotation, they are paired with an experienced engineer for their first two shifts. Both the experienced engineer and the person onboarding are paid in full during this Co-pilot phase. I find this to be a great example of onboarding and having the right incentives in place.

I asked Anna if the company ever considered purely team-based oncalls. She said that they did, but they could not see it being workable without increasing the load on engineers several times over.

I found it curious that both Intercom and LaunchDarkly ended up in the same place with regards to oncall. Anna told me that this was not just a lucky accident:

"We had a former Intercom engineer take part in developing the new oncall process here, and they advocated for some of the practices they'd seen work well. Pretty incredible to have an engineer championing their former company's oncall. Although we ended up with a slightly different model, we definitely took note and I want to make sure to acknowledge that."

TravelPerk – disclaimer: [I'm an advisor here](#) – has an oncall practice that caught my attention. They put a "less intense" oncall rotation in place. All of the company's engineering team is based in Europe, split between offices in Spain and the UK. The oncalls are divided up like this:

- **L1:** 7am to 11pm coverage. All alerts come in here during non-nighttime hours. A paid rotation at around €420/week, with different amounts for weekdays and

weekends.

- **L2:** 24/7 coverage. L1 alerts are escalated here, and go straight to here during night hours. Pays €500/week.
- **L3 / DevOps** coverage at 24/7. L2 alerts escalate to this rotation. Pays €320/week.

What I find curious about this setup is that the primary oncall (L1) is never paged during nighttime: there's a separate oncall to which such alerts go. Also, the third line of defense is not engineering managers like at most companies, but is a dedicated DevOps rotation. I talked with an engineer at the company who summarized it like this:

"During the daytime, L1 receives the pager. If not acknowledged, it escalates to L2. Both L1 and L2 manually escalate to DevOps if the issue is infra-related. Alerts usually never reach L2, and much less DevOps. It's a nice setup overall."

I reached out directly to Chief Product and Technology Officer, [Ross McNairn](#), for more context on why they chose such a setup. He shared:

"After 11pm, coverage deliberately drops, and we only take action on the most serious things. So while we do have a 24/7 oncall, it is, by design, "less intense." If something major is triggered during the night, it hits the L2 and they can escalate further to L3 if needed.

We are now looking into how we can further improve our oncall: could a non-engineering support and incident management team might replace L1? If we could get there, we could further reduce the need for engineering oncall outside office hours."

Monzo is an online bank in the UK, and a company with [more than 1,500 microservices](#). The firm shared how it did oncall [in 2018](#) and recently [published](#) a follow-up to this. The company combines several approaches we've seen at the above companies:

- **Two rotations to distribute work fairly.** Monzo has an in-hours rotation (10am-6pm weekdays) and an out-of-hours one. Engineers need to acknowledge alerts within 15 minutes on each.

- **Paid, voluntary oncall outside hours.** The company pays both the primary and secondary oncall engineers £500 per week.
- **Second line of defense with specialists.** The first line of defense is the voluntary oncall rotation. This rotation then reaches out to specialist oncalls like payments specialists, security specialists, and so on. These oncalls are more relaxed, with a 1 hour acknowledgement time and are paid £300 per week. The specialist oncall is also voluntary, and the page load is much lower.

The goal for Monzo was to balance reliability with a human-centered oncall approach. I recommend [reading their deepdive for more details](#).

6. Healthy oncall practices

What are healthy oncall practices? Here are some which stand out to me as ones that could create a healthy oncall:

Expect the people oncall to do no critical work, nor work on the roadmap, but to improve oncall instead. A practice I've observed at several companies is that being oncall is paired with doing oncall improvements. Doing such work acknowledges that oncall can be disruptive for focused work, and it takes the pressure off delivering as usual, while adding the expectation of acting on any incoming alerts.

Software engineer [Olivier Suritz](#) – formerly of Google, now at Coinbase – investigated how to arrange oncall rotations in order to reduce toil and improve developer experience. This is what he shared:

"What I experienced as the most calm rotations is to have the secondary oncall working on tasks specifically meant to improve the toil. For example, fixing bugs that cause alerting, slowly improving the oncall infrastructure. Improve infrastructure, improve automated testing etc to catch errors earlier on so you don't get paged at 3am.

Fixit weeks that are sponsored by senior management are also a good approach. This allows for finally spending a week just fixing bugs and infra issues."

Compensate for going oncall; offer cash or vacation. Working outside of core hours, and arranging your evenings, nights, weekends and holidays to be on standby, is an additional ask of everyone. Be respectful of the fact people are working more and offer to compensate them for the additional time and stress which comes with oncall.

Offer cash and – ideally – the ability to choose vacation, instead.

At companies where every engineer is oncall, you'll find that such an approach is not financially viable. My response is that these places likely have too many engineers oncall, and burn them out more than necessary. I suggest [looking into why Intercom moved away from this model](#), and the benefits they saw as a result.

Other companies will say they pay engineers enough to take on this additional load. I suggest these companies look to Google, and ponder why Google not only almost certainly pays more than they do, but also compensates generously for oncall.

If you don't pay for oncall, don't be surprised if oncall is a reason for attrition. As a current engineer at publicly traded tech company valued at \$17B shared:

"My company does not do not pay anything additional for being oncall. I hate oncall life and once it's time to move on, I would prefer no oncall. I'm oncall every 4 weeks and it's getting to me."

And oncall was a reason why an engineer left another publicly traded tech company, this one valued at \$2.5B:

"I had never done oncall before I worked at this place, and it was my only real hesitation about joining the company. As it turns out, it was one of the main reasons I ended up leaving. The oncall for my team didn't seem terrible, but I have zero interest in having to be ready to jump online at all hours of the day. If I were paid for my time, my feelings might've changed."

Even at places with great engineering culture, oncall can get in the way of internal mobility. As a current engineer at a public tech company valued at \$15B shares:

"Some teams have had dramatically different pager volumes for years at HubSpot. There are teams I'd really want to work on thanks to their great mission, but those same teams have a noisy oncall. There is no way I'm doing this with so much unpaid overtime at anti-social hours. I want to have a life outside of work."

Volunteer-only, paid, shared oncalls. The approach that Intercom and LaunchDarkly adopted might sound too good to be true; volunteer-only oncalls which work well. But both companies are happy with the end result.

In order to build such a setup, a few things help:

- **Pay** an attractive rate to be oncall. This helps both with keeping this rota volunteer-only, and also acts as a nice reward for those taking on the additional responsibility.
- **Invest** in improving the oncall, especially when putting the system in place. Investing may mean standardizing alerting and monitoring, teams putting runbooks together, making expectations clear, and so on.
- **Iterate** on the setup regularly. As teams grow, a new type of setup might make sense, especially as it's no longer backend teams, but web or mobile teams, who might define their own oncall approaches.

Shared oncalls are a straightforward way to ease the load on any engineering team. The beauty of this approach is that it can be done at a team level. So, if you're an engineer or an engineering manager, you might be able to execute on such an initiative.

A software engineer at Dutch telco provider KPN shared:

"In my area, several teams share an oncall. We work together and have guidelines in place on where to find operation information and log in a standardized way, report errors similarly, and so on. Thanks to so many similarities, it's easy for teams to share rotations. In my case, I'm oncall once every 2-3 months thanks to the shared oncalls."

KPN – like many telco companies – are less on the cutting edge compared to startups and some of Big Tech. However, if a company like KPN can – and does! – share oncalls,

what is stopping teams in your organization from doing so?

Use follow-the-sun oncalls to reduce workload. If teams are spread across different time zones, take advantage of this and attempt to rely on engineers in another timezone to respond to alerts during their regular business hours.

In practice, few teams are truly distributed across time zones, and this is where sharing oncalls works well. If Team A is based in one timezone, and Team B is 8 hours behind, and those two teams can share an oncall, then each oncall engineer needs to spend only 8 hours per day oncall, instead of 16 without the sharing. They need to go oncall half as frequently, thanks to merging their oncalls.

Red Hat, LaunchDarkly and Intercom all follow this approach.

Minimize out-of-hours alerts and measure this. The only time alerts should ring outside of business hours is when there's an outage which needs mitigating. If the alerts fired are noisy, make sure to act on fixing the root cause of them.

Limit how often engineers can go oncall. I'm surprised so few companies do what Google does of setting a quota on how often it's okay for engineers to be oncall, and then designing a system that implements this.

For startups, I am empathetic with the hustle that's expected for success, and that many of these places are scrappy, without legions of software engineers. However, the startup phase is the perfect time to put healthy oncall practices in place, which will grow with the company.

If oncall is not voluntary, set the cap of what "healthy" oncall duties are per quarter, then measure this and make people doing more of this duty the exception, not the rule.

Don't prioritize product work for those doing oncall shifts. Being oncall is stressful enough; you add to the stress by expecting product work to be done to the usual schedule.

Some companies encourage people to deprioritize product work or to fully stop working on it, should they get an alert. If it's an outage, they should commit their time

to mitigating it, then fixing systemic issues to prevent similar outages from happening. And if it was a false alert, the oncall engineer should improve the alerting infrastructure.

Another approach I've observed is the oncall engineer being expected to work on smaller, customer-facing issues or complaints, as a "support engineer" role. This is typically work which doesn't require long periods of focus time. In this case, the expectation is also to prioritize oncall-related work.

Minimize the impact of people's nights being disrupted. There is hardly anything worse than being woken up in the dead of night, *especially* if the alert is not for an outage. So don't do it, unless there's a very, very good reason!

Utilize follow-the-sun rotations where possible. If you don't have this option, consider doing what TravelPerk did: specify the primary oncall rotation is to not be woken up at night, and only alert for breaking issues during this period.

It's helpful to have a rotation that's never woken up in the middle of the night, and one that can only be paged during this time. This is because in the event of a series of alerts, it allows people to take over shifts from each other and to rest without interruption.

Specify the number of hours of rest after working on an incident. Most companies tell people it's okay to start work later if they had a nighttime incident to deal with. However, what often happens is people usually get up and start work as normal the next day, due to feeling guilty.

In Germany, it's legally required for people to have 11 hours of rest between shifts. So if someone works on an outage at 2am, they cannot start work before 1pm. Consider making similar, explicit expectations; for example, tell people they should rest at least 8 hours after being woken up by a page.

Setting such limits to reduce the time people work when paged, will also help prioritize more investment in oncall.

Differentiate tiers of oncall. Does every service need alerts to be responded to within 5 minutes? The answer to this will definitely be "no." Some systems and services are

more critical than others; oncall coverage should reflect this reality.

Define a tiering system for oncall coverage: what are critical systems? What are nice-to-haves? With such a system in place, you can draw up a more efficient oncall strategy.

Oncall for critical systems should be organized in a way that has good coverage, and which pays in cash or vacation for responding to alerts within minutes.

Take inspiration from how Google defines Tier 1, Tier 2 and Tier 3 alerts. This is a sensible system that's a good starting point for most companies.

Consider suppressing Tier 2 and Tier 3 alerts during nighttimes and not waking up engineers for alerts which can wait until morning.

I've observed that defining such tiering and then rolling it out, is a longer and more complicated process than most people expect. Still, until you do it, you are at risk of having a noisy alerting system that burns engineers out more than it should.

Make it clear oncall is part of the job when it's mandatory for most people. Most companies that expect people to go oncall don't mention this in their job adverts. This can create a mismatch in expectations.

Instead, follow the example of companies like DuckDuckGo, which openly states oncall is part of the job. Like [in this job advert](#):

"What you will do: (...)

Participate in our oncall rotation; triage and address reliability issues that come up in production."

Lead by example. As an engineering manager or tech lead, make a point to go oncall for some rotations. This way, you experience how good or bad the rotation is, and you can champion initiatives which improve being oncall for everyone.

7. Regulations across countries

When setting up oncall rotations, there are countries where local regulation will shape compensating oncall, place constraints on how much someone can go oncall, or mandate rest periods after responding to an incident outside hours. On the other hand, there are countries which place few to no restrictions in the form of regulation.

I did research by asking people in my network on the policies their organization has in place, and cross-checking with publicly accessible regulations, online.

Treat this list as a starting point, and please mention in the comments any countries I missed, and I'll add them here. *Note that this list is likely to be incomplete, and won't be a substitute for confirming how local regulation evolves after publication.*

Countries that regulate oncall in ways that are relevant for software engineers

Brazil: pay. Oncall standby compensation is regulated to be paid as a third of normal hourly wages. For example, at Amazon in Brazil, engineers need to submit the hours they've worked oncall and they receive additional compensation for that month.

If an engineer is paid and needs to do work to mitigate an incident, they are entitled to 150% of their hourly wage for those hours worked. Working at night – between 10pm and 5am – [entitles](#) engineers to an at least 20% premium to their normal hourly wages, which means 170% of their hourly wage (150% because it's overtime work, plus 20% because of nighttime work).

France: pay or rest time. France regulates being on standby oncall as working time. From an article on French Business Advice:

“The Labor Code (art. L. 3121-9) defines it as follows: ‘A period during which the employee, without being at his place of work and without being at the permanent and immediate disposal of the employer, must be able to intervene to accomplish work in the service of the company.’ The duration of this work is considered as effective working time.

The oncall time presupposes that the employee is not present at the workplace.

The oncall period necessarily entails the granting of compensation either in financial form or in the form of rest. The amount or volume of this compensation is set by the collective agreement or, in the event of a unilateral set up, by the employer."

France is strict in limiting working time so as to not exceed an average of 44 hours per week for 12 consecutive weeks, and mandates a minimum daily rest of 11 consecutive hours.

The French legislation was drafted more with doctors or firefighters in mind, so most French companies tend to ignore the rest time part, and offer a fixed rate for employees going oncall.

In France, the [Right to Disconnect regulation](#) passed in 2016 in theory makes implementing an oncall even more challenging. In practice, companies offering a fixed rate usually don't have problems with organizing oncall.

Spain: pay. Spanish law differentiates between two kinds of oncall. As per law firm Mariscal & Abogados [interpreting a 2017 lawsuit](#) related to oncall compensation where a worker was required to respond to pages within eight minutes, this setup is considered as working time:

"If oncall service requires physical presence, either in a workplace or in a place determined by the employer, and the worker must be available to provide services for the employer if needed, it is considered working time.

On the other hand, a system of oncall service where a worker has to be permanently available, however, without the obligation to be present in a workplace, does not imply working time. That is because a worker, in these circumstances, can manage his time with fewer limitations and devote himself to his interests. In this case, only the time dedicated to the effective provision of services should be considered working time."

All companies in Spain compensate for oncall time, ranging from €250 to €1,000 per week, as shared in this [oncall compensation models in Spain](#) spreadsheet which was sourced by software engineers working in Spain.

Several companies in Spain, on top of paying for standby, pay for incidents separately, where people have to work on mitigating them.

Germany: rest time. German law requires a minimum of 11 hours of uninterrupted rest time at the end of a working day. Getting paged, and having to work on resolving an incident counts as the working day. Therefore, if getting paged at 2am and having to do work, the employee is entitled to come to the office at or after 1pm, and still leave at the usual time.

German regulation does not count standby oncall as work. As per this [Employee's Rights in Germany](#) article:

"Oncall work in the form of constant availability, for example stand-by work, on the other hand, does not count as working time."

Even though pay for oncall is not regulated, a surprising number of companies with headquarters in Germany pay generous oncall compensation for software engineers.

Switzerland: pay plus limiting time spent oncall. In Switzerland, oncall needs to be compensated. As per the Oncall Work In Switzerland article [by Mondaq](#):

"In a court ruling of 1998, the Swiss Federal Supreme Court for the first time confirmed the admissibility of oncall work. However, it stated that oncall time must be compensated, but not necessarily at the same rate as normal pay. As a general rule, the less free the employee is during oncall time, the higher the compensation must be. Another test is the employer's notice period: the shorter the notice period, the more limited is the employee's private life and therefore a higher compensation must be paid.

This compensation can already be included in the salary and does not have to be remunerated additionally. For such an agreement to be valid, the respective salary must be higher than the one for employees that are not oncall workers."

Oncall pay can be skipped if the employees oncall already get a higher salary than those not oncall.

Oncall duty is limited to seven days per month. From the [Work and rest period](#) regulation:

"An employee may not be required to work on call for more than seven days in any period of four weeks. After the last oncall duty has ended, the employee may not be required to work on call for the following two weeks.

The employee may exceptionally be required to work on call for no more than 14 days in a period of four weeks but may not be called in more than five times per month on average over a calendar year."

Sweden: rest time. As per the [Working Hours Act](#), employees are entitled to 36 hours of rest for every 7 days worked:

"According to the Act, employees have a right to an uninterrupted weekly rest of at least 36 hours for each 7 day period. If it's possible, the weekly rest should be on the weekend."

This regulation is why most Swedish companies run oncall shifts from Thursday to Thursday. When oncall, the employee is not considered resting, and is on standby through the weekend. For the next week, they are now entitled to 72 hours – or 3 days – of uninterrupted rest. This is why they get the Friday off, as well as the weekend.

As a note, collective agreements can override the above setup in Sweden.

Austria: rest time plus limiting time spent oncall. Austria, similar to Germany, [mandates](#) 11 hours of rest after working, though when oncall, the Austrian law [reduces](#) this to 8 hours. Austria also mandates an uninterrupted rest period of 36 hours during the weekend, similar to Sweden. This means that if doing an oncall shift over the weekend, the next week, employees are entitled to 72 hours of rest. Austria limits oncall duty to ten days per month.

Regulation does not specify compensation oncall, but collective agreements can specify this. For example, the [collective agreement for employees of service providers in the field of information technology](#) – which should apply to most software engineers in Austria, who qualify as IT workers – specifies this:

"A maximum of 10 oncall duties is allowed per month (up to a maximum of 168 hours). Within a 3-month period, oncall duty may only be agreed on 30 days. The lump sum for oncall duty is €4.43 per hour for the period of the agreed oncall duty. As soon as the oncall employee is called upon to work, working hours start. oncall duty must be agreed in writing in due time."

Poland: pay when paged and limits to being oncall. As per the [oncall duty regulation](#) of the Ministry of Family and Social Policy:

"Oncall time is counted as working time only if the employee actually worked while on duty. On the other hand, if the employee did not work while on duty, the oncall time is not counted as working time.

oncall time must not affect the employee's right to daily and weekly rest. This means that it is not permissible to instruct the employee to be on call for 24 hours or a whole week. The total oncall time and working time must not exceed 13 hours per day, and the employee must also have a minimum of 35 hours of continuous weekly rest. This also applies to oncall time at home. However, these restrictions do not apply to employees managing the workplace on behalf of the employer."

Czech Republic: pay. The country [mandates](#) at least 10% of the hourly wage to be paid when oncall and at least 25% when working outside normal hours.

Hungary: pay plus limiting time spent oncall. Hungary [mandates](#) paying 20% of the hourly salary for being standby oncall and 150% of the salary for doing work during this time; for example, when paged, and mitigating an incident. The maximum amount of oncall per month is 7 days (168 hours).

To summarize: around the clock oncall for seven days a week is not allowed for employees, and such oncall needs to be organized in shifts. This excludes managers, who can be put on such oncall.

Italy. Being on standby oncall is not considered as working hours. As law firm Osborne Clark [writes](#):

"Regulations on normal working time are not applicable to employees performing discontinuous or standby duties."

However, most collective contracts that are in place in Italy do specify oncall expectations. Many companies in Italy use [this collective agreement contract](#) which specifies a fixed sum for being on standby - which is defined based on pay grade. If the oncall engineer is paged and needs to do work outside business hours, the usual overtime regulations apply.

Countries that regulate oncall, but exempt most software engineers

Several countries have oncall regulations. However, many of these don't apply to software engineers, thanks to this profession typically earning far above the minimum wage.

India regulated overtime work as part of its [Factories Act](#) in 1948. This law states:

"No employee is supposed to work for more than 48 hours in a week and 9 hours in a day. Any employee who works for more than this period is eligible for overtime remuneration prescribed as twice the amount of ordinary wages."

However, this regulation was drafted with hourly-paid workers in mind, so most tech companies ignore it. Another regulation that is more relevant is the [Minimum Wages Act](#) from 1950. This legislation also specifies overtime compensation, stating:

"When a worker works in an employment for more than nine hours on any day or for more than forty-eight hours in any week, he shall in respect of overtime work, be entitled to wages - in the case of any other scheduled employment, at double the ordinary rate of wages;"

In 1950, there was no concept of standby oncall, so most employers today ignore this regulation. Instead of paying for being oncall, it's more common for employers to offer rest time after oncall shifts.

The Netherlands regulates oncall pay and rest time as part of the [Working Hours Act](#). However, employees earning three times the minimum wage are exempt. This limit is around €50,000 a year, meaning that most software engineers will be outside this regulation.

Countries that don't regulate oncall for software engineers

Below are countries which don't have specific regulations that apply to software engineers being oncall. The one regulation that applies to all of the countries below is minimum wage. This means that if software engineers are paged and have to work outside of working hours, their bi-weekly or monthly wage should be the minimum hourly wage multiplied by the hours worked. For software engineers, being paid above minimum wage even if counting hours spent oncall is almost always the case though.

The US has the [FLSA](#) (Fair Labor Standards Act) which specifies definitions relevant for oncall. This act specifies Waiting Time, and differentiates between two states:

- Engaged to wait: considered as work time. An example is a fireman who plays checkers while waiting for an alarm.
- Waiting to engage: not considered as working time. Software engineers being oncall are considered in this category, for the most part.

In the past, federal court decisions have held that oncall employees are not overly constrained by a paging device, publication SHRM [wrote](#). Because oncall is considered waiting to engage, no additional requirements are for oncall, save for ensuring that if working – e.g. resolving an incident – then that counts as working time, and employees need to be paid at least minimum wage when summarizing all working hours.

Canada does not regulate oncall pay. While the [Canada Labour Standards Regulations](#) defines expectations for reporting pay, this provision does not apply to oncall which is considered stand-by. As per [Reporting Pay - 802-1-IPG-048](#):

"The fact that an employee is "on call" or on "stand-by" outside regular working hours is not enough to trigger the reporting pay provision. An employee must actually report to his or her workplace in order to activate the section."

However, companies can draft a collective agreement, or add stand-by premiums to company policy. For example, this is what the [City of Toronto has implemented](#), paying 15% of hourly wages for being oncall:

"Employees who are required to be on standby status, whether or not they are called into work are eligible for standby pay. Employees are paid one hour for every eight hours or less, while on standby status, on a straight time basis."

The UK has a minimum wage law, and every worker needs to be paid this minimum wage, taking into account all working hours. In 2018, [a judge ruled](#) that being oncall or on standby counts as working hours. Therefore, if someone is oncall for a week, they need to be paid 168 hours times the UK minimum wage (£8.91 per hour, or £1,496 for that week). Given that most software engineers make far more than the minimum wage, they already make more, even if oncall, than the hours worked multiplied by this minimum hourly wage.

Takeaways

The highest performing engineering teams have autonomy to make their own decisions, and own these decisions. This is only possible if those same engineers take responsibility when things go wrong. This, in turn, involves those same engineers being oncall. In this way, being oncall is a price you pay to be more nimble and have more autonomy as an engineering team.

Oncall comes with plenty of downsides in the mid and long-term, in the form of stress, pressure and burnout.

I've observed most of Big Tech except Google ignore the very real risk of oncall burnout. This is a mistake because heavy oncalls will both result in attrition, and can also cause unnecessary stress.

If you work at a company where small teams own all of their oncall duties and people are sometimes paged in the middle of the night, know that your practices are anything but healthy. Look at examples of what companies like Google, Intercom, LaunchDarkly, TravelPerk or Monzo have done to ease oncall load.

This issue summarized healthy oncall practices. However, these healthy practices start by acknowledging that oncall *does* count as additional work and stress, and that there are ways to make this work more bearable for everyone. In some cases, you can have the oncall rotation staffed exclusively by volunteers, creating a win-win for everyone.

In next week's issue we'll lift the lid on the numbers and go into detail on exactly how much companies pay engineers for going oncall, and how they structure this compensation.

Thanks to [Anna Baker](#), [Andrea Borgia](#), [Kevin Meyer](#) and [Ryan Sherlock](#) for contributing to this article.

How did you like this issue? 🤔

[Amazing](#) • [Great](#) • [Good](#) • [OK](#) • [So-so](#)



84 Likes

9 Comments



Write a comment...



Abhijeet Jul 26, 2022 ❤️ Liked by Gergely Orosz

This is a poorly understood facet of a software engineering job. Thank you for this deep dive.

♡ LIKE (3) 💬 REPLY ...



Peter Jul 28, 2022 ❤️ Liked by Gergely Orosz

Amazing timing Gergely! I am currently working on renewing the oncall system at our company and then you post this great article with a lot to take away :) Thank you!

 LIKE (1)  REPLY ...

1 reply by Gergely Orosz

7 more comments...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing